

YOLO5Face: Why Reinventing a Face Detector

Delong Qi, Weijun Tan*, Qi Yao, Jingfeng Liu

Shenzhen Deepcam Information Technologies

Shenzhen, China

{delong.qi, weijun.tan, qi.yao, jingfeng.liu}@deepcam.com

*LinkSprite Technologies, USA, weijun.tan@linksprite.com

Abstract—Tremendous progress has been made on face detection in recent years using convolutional neural networks. While many face detectors use designs designated for the detection of face, we treat face detection as a general object detection task. We implement a face detector based on YOLOv5 object detector and call it YOLO5Face. We add a five-point landmark regression head into it and use the Wing loss function. We design detectors with different model sizes, from a large model to achieve the best performance, to a super small model for real-time detection on an embedded or mobile device. Experiment results on the WiderFace dataset show that our face detectors can achieve state-of-the-art performance in almost all the Easy, Medium, and Hard subsets, exceeding the more complex designated face detectors. The code is available at <https://www.github.com/deepcam-cn/yolov5-face>.

Index Terms—Face detection, convolutional neural network, YOLO, real-time, embedded device, object detection

I. INTRODUCTION

Face detection is a very important computer vision task. Tremendous progresses have been made since deep learning, particularly convolutional neural network (CNN), has been used in this task. As the first step of many tasks, including face recognition, verification, tracking, alignment, expression analysis, face detection attracts many researches and developments in the academia and the industry. And the performance of face detection has improved significantly over the years. For a survey of the face detection, please refer to the benchmark results [1], [2]. There are many methods in this field from different perspectives. Research directions include design of CNN network, loss functions, data augmentations, and training strategies. For example, in the YOLOv4 paper, the authors explore all these research directions and propose the YOLOv4 object detector based on optimizations of network architecture, selection of bags of freebies, and selection of bags of specials [3].

In our approach, we treat the face detection as a general object detection task. We have the same intuition as the TinaFace [4]. Intuitively, face is an object. As discussed in the TinaFace [4], from the perspective of data, the properties that faces has, like pose, scale, occlusion, illumination, blur and etc., also exist in other objects. The unique properties in faces like expression and makeup can also correspond to distortion and color in objects. Landmarks are special to face, but they are not unique either. They are just key points of an object. For example, in license plate detection, landmarks are also used. And adding landmark regression in the object prediction head is straightforward. Then from the perspective

of challenges encountered by face detection like multi-scale, small faces and dense scenes, they all exist in generic object detection. Thus, face detection is just a sub task of general object detection.

In this paper, we follow this intuition and design a face detector based on the YOLOv5 object detector [5]. We modify the design for face detection considering large faces, small faces, landmark supervision, for different complexities and applications. Our goal is to provide a portfolio of models for different applications, from very complex ones to get the best performance to very simple ones to get the best trade-off of performance and speed on embedded or mobile devices.

Our main contributions are summarized as following,

- We redesign the YOLOv5 object detector [5] as a face detector, and call it YOLO5Face. We implement key modifications to the network to improve the performance in terms of mean average precision (mAP) and speed. The details of these modifications will be presented in Section III.
- We design a series of models of different model sizes, from large models, to medium models, to super small models, for needs in different applications. In addition to the backbone used in YOLOv5 [5], we implement a backbone based on ShuffleNetV2 [6], which gives the state-of-the-art (SOTA) performance and fast speed for mobile device.
- We evaluate our models on the WiderFace [1] dataset. On VGA resolution images, almost all our models achieve the SOTA performance and fast speed. This proves our goal, as the title of this paper claims, we do not need to reinvent a face detector since the YOLO5Face can accomplish it.

II. RELATED WORK

A. Object Detection

General object detection aims at locating and classifying the pre-defined objects in a given image. Before deep CNN is used, traditional face detection uses hand crafted features, like HAAR, HOG, LBP, SIFT, DPM, ACF, etc. The seminal work by Viola and Jones [7] introduces integral image to compute HAAR-like features. For a survey of face detection using hand crafted features, please refer to [8], [9].

Since the deep CNN shows its power in many machine learning tasks, face detection is dominated by deep CNN

methods. There are two-stage and one-stage object detectors. Typical two-stage methods are the RCNN family, including RCNN [10], fast-RCNN [11], faster-RCNN [12], mask-RCNN [13], Cascade-RCNN [14].

The two-stage object detector have very good performance but suffers from long latency and slow speed. In order to overcome this problem, one-stage object detectors are studied. Typical one-stage networks include SSD [15], YOLO [3], [5], [16]–[18].

Other object detection networks include FPN [19], MMDetection [20], EfficientDet [21], transformer(DETR) [22], Centernet [23], [24], and so on.

B. Face Detection

The researches for face detection follows the general object detection. After the most popular and challenging face detection benchmark WiderFace dataset [1] is released, face detection develops rapidly focusing on the extreme and real variation problem including scale, pose, occlusion, expression, makeup, illumination, blur and etc.

A lot of methods are proposed to deal with these problems, particularly the scale, context, anchor in order to detect small faces. These methods include MTCNN [25], FaceBox [26], S3FD [27], DSFD [28], RetinaFace [29], RefineFace [30], and the most recent ASFD [31], MaskFace [32], TinaFace [4], MogFace [33], and SCRFD [34]. For a list of popular face detectors, the readers are referred to the WiderFace website [2].

It is worth noting that some of these face detectors explore unique characteristics in human face, the others are just general object detector adopted and modified for face detection. Use RetinaFace [29] as an example, it uses landmark (2D and 3D) regression to help the supervision of face detection, while TinaFace [4] is simply a general object detector.

C. YOLO

YOLO first appeared in 2015 [16] as a different approach than popular two-stage approaches. It treats object detection as an regression problem rather than a classification problem. It performs all the essential stages to detect an object using a single neural network. As a result, it not only achieves very good detection performance, but also achieves real-time speed. Furthermore, it has excellent generalization capability, can be easily trained to detect different objects.

Over the next five years, the YOLO algorithm have been upgraded to five versions with many innovative ideas from the object detection community. The first three versions - YOLOv1 [16], YOLOv2 [17], YOLOv3 [18] are developed by the author of the original YOLO algorithm. Out of these three versions, the YOLOv3 [18] is a milestone with big improvements in performance and speed by introducing multi-scale features (FPN) [19], better backbone network (Darknet53), and replacing the Softmax classification loss with the binary cross-entropy loss.

In early 2020, after the original YOLO authors withdrawn from the research field, YOLOv4 [3] was released by a different research team. The team explore a lot of options in

almost all aspects of the YOLOv3 [18] algorithm, including the backbone, and what they call bags of freebies, and bags of specials. It achieves 43.5% AP (65.7% AP50) for the MS COCO dataset at a real time speed of 65 FPS on Tesla V100.

One month later, the YOLOv5 [5] was released by another different research team. In the algorithm prospective, the YOLOv5 [5] does not have many innovations. And the team does not publish a paper. These bring quite some controversies about if it should be called YOLOv5. However, due to its significantly reduced model size, faster speed, and similar performance as YOLOv4 [3], and a full implementation in Python (Pytorch), it is welcome by the object detection community.

III. YOLO5FACE FACE DETECTOR

In this section we present the key modifications we make in YOLOv5 and make it a face detector - YOLO5Face.

A. Network Architecture

We use the YOLOv5 object detector [5] as our baseline and optimize it for face detection. We introduce some modifications designated for detection of small faces as well as large faces.

The network architecture of our YOLO5Face face detector is depicted in Fig. 1. It consists of the backbone, neck, and head. In YOLOv5, a new designed backbone called CSPNet [5] is used. In the neck, an SPP [35] and a PAN [36] are used to aggregate the features. In the head, regression and classification are both used.

In Fig. 1 (a), the overall network architecture is depicted. In Fig. 1 (b), a key block called CBS is defined, which consists of Conv layer, BN layer, and a SiLU [37] activation function. This CBS block is used in many other blocks. In Fig. 1 (c), an output label for the head is shown, which include bounding box (bbox), confidence (conf), classification (cls) and five-point landmarks. The landmarks are our addition to the YOLOv5 to make it a face detector with landmark output. If without the landmark, the last dimension 16 should be 6. Please note that, the output dimensions $80*80*16$ in P3, $40*40*16$ in P4, $20*20*16$ in P5, $10*10*16$ in optional P6 are for every anchor. The the real dimension should be multiplied by the number of anchors.

In Fig. 1 (d), a Stem structure [38] is shown, which is used to replace the original Focus layer in YOLOv5. The introduction of the Stem block into YOLOv5 for face detection is one of our innovations.

In Fig. 1 (e), a CSP block (C3) is shown. This block is inspired by the DenseNet [39]. However, instead of adding the full input and the output after some CNN layers, the input is separated two two halves. One half is passed through a CBS block, a number of Bottleneck blocks, which is shown in Fig. 1 (f), then another Conv layer. The other half is passed through a Conv layer, then the two are concatenated, followed by another CBS block.

Fig. 1 (g), an SPP block [35] is shown. In this block the three kernel sizes $13*13$, $9*9$, $5*5$ in YOLOv5 are revised to $7*7$, $5*5$, $3*3$ in our face detector. This has been shown

as one of the innovations that improves the face detection performance.

Note that we only consider VGA resolution input images. To be more precise, the longer edge of the input image is scaled to 640, and the shorter edge is scaled accordingly. The shorter edge is also adjusted to be a multiple of the largest stride of the SPP block. For example, when P6 is not used, the shorter edge needs to be multiple of 32; when P6 is used, the shorter edge needs to be multiple of 64.

B. Summary of Key Modifications

The key modifications are summarized as follows.

- We add a landmark regression head to the YOLOv5 network. The Wing loss [40] is used as a loss function for it. This makes the face detector more useful since landmarks are used in many applications. The landmark locations are more accurate. This extra supervision helps the face detector accuracy.
- We replace the Focus layer of YOLOv5 [5] with a Stem block structure [38]. It increases the network's generalization capability, and reduces the computation complexity while the performance does not degrade.
- We change the SPP block [35] and use a smaller kernel. It makes the YOLOv5 more suitable for face detection and improve the detection accuracy.
- We add a P6 output block with stride of 64. It increases the capability to detect large faces. This is an item easily overlooked by many researchers since their focuses are more on the detection of small faces.
- We find that some data augmentation methods on general object detection are not appropriate on face detection, including up-down flipping and Mosaic. Removing the up-down flipping improves the performance. When small images are used, the Mosaic augmentation [3] degrades the performance. However, when the small faces are ignored, it works well. Random cropping helps the performance.
- We design two super light-weight models based on ShuffleNetV2 [6]. This backbone is very different from the CSP network. These models are super small, while achieve SOTA performance for embedded or mobile device.

C. Landmark Regression

Landmarks are important characteristics for human face. They can be used to do face alignment, face recognition, face express analysis, age analysis etc. Traditional landmarks consist of 68 points. They are simplified to 5 points in MTCNN [25]. Since then, the five-point landmarks have been used widely in face recognition. The quality of landmarks affects the quality of face alignment and face recognition.

The general object detector does not include landmarks. It is straightforward to add it as a regression head. Therefore, we add it into our YOLO5Face. The landmark outputs will be used in align face images before they are sent to the face recognition network.

General loss functions for landmark regression are L2, L1, or smooth-L1. The MTCNN [25] uses the L2 loss function. However, it is found these loss functions are not sensitive to small errors. To overcome this problem, the Wing-loss is proposed [40],

$$wing(x) = \begin{cases} w \cdot \ln(1 + |x|/e), & \text{if } x < w \\ |x| - C, & \text{otherwise} \end{cases} \quad (1)$$

The non-negative w sets the range of the nonlinear part to $(-w, w)$, e limits the curvature of the nonlinear region and $C = w - w \ln(1 + w/e)$ is a constant that smoothly links the piecewise-defined linear and nonlinear parts. Plotted in Fig. 2 is this Wing loss function with different parameters w and e . It can be seen that the response at small error area near zero is boosted compared to the L2, L1, or smooth-L1 functions.

The loss functions for landmark point vector $s = \{s_i\}$, and its ground truth $s' = \{s'_i\}$, where $i = 1, 2, \dots, 10$, is defined as,

$$loss_L(s) = \sum_i wing(s_i - s'_i) \quad (2)$$

Let the general object detection loss function of YOLOv5 be $loss_O(bounding_box, class, probability)$, then the new total loss function is,

$$loss(s) = loss_O + \lambda_L \cdot loss_L \quad (3)$$

where the λ_L is a weighting factor for the landmark regression loss function.

D. Stem Block Structure

We use a stem block similar to [38]. The stem block is shown in Fig. 1 (d). With this stem block, we implement a stride = 2 in the first spatial down-sampling on the input image, and increase the number of channels. With this stem block, the computation complexity only increase marginally, while a strong representation capability is ensured.

Model	Backbone	(D,W)	With P6?
YOLOv5s	YOLO5-CSPNet [5]	(0.33,0.50)	No
YOLOv5s6	YOLO5-CSPNet	(0.33,0.50)	Yes
YOLOv5m	YOLO5-CSPNet	(0.50,0.75)	No
YOLOv5m6	YOLO5-CSPNet	(0.50,0.75)	Yes
YOLOv5l	YOLO5-CSPNet	(1.0,1.0)	No
YOLOv5l6	YOLO5-CSPNet	(1.0,1.0)	Yes
YOLOv5n	ShuffleNetv2 [6]	-	No
YOLOv5n-0.5	ShuffleNetv2-0.5 [6]	-	No

TABLE I
DETAIL OF IMPLEMENTED YOLO5FACE MODELS, WHERE (D,W) ARE THE DEPTH AND WIDTH MULTIPLES OF THE YOLOV5 CSPNET [5]. THE NUMBER OF PARAMETERS AND FLOPS ARE LISTED IN TABLE III.

E. SPP with Smaller Kernels

Before forwarding to feature aggregation block in the neck, the output feature maps of the YOLO5 backbone are sent to an additional SPP block [35] to increase the receptive field and separate out the most important features. Instead of many CNN models containing fully connected layers which only accept input images of specific dimensions, SPP is proposed

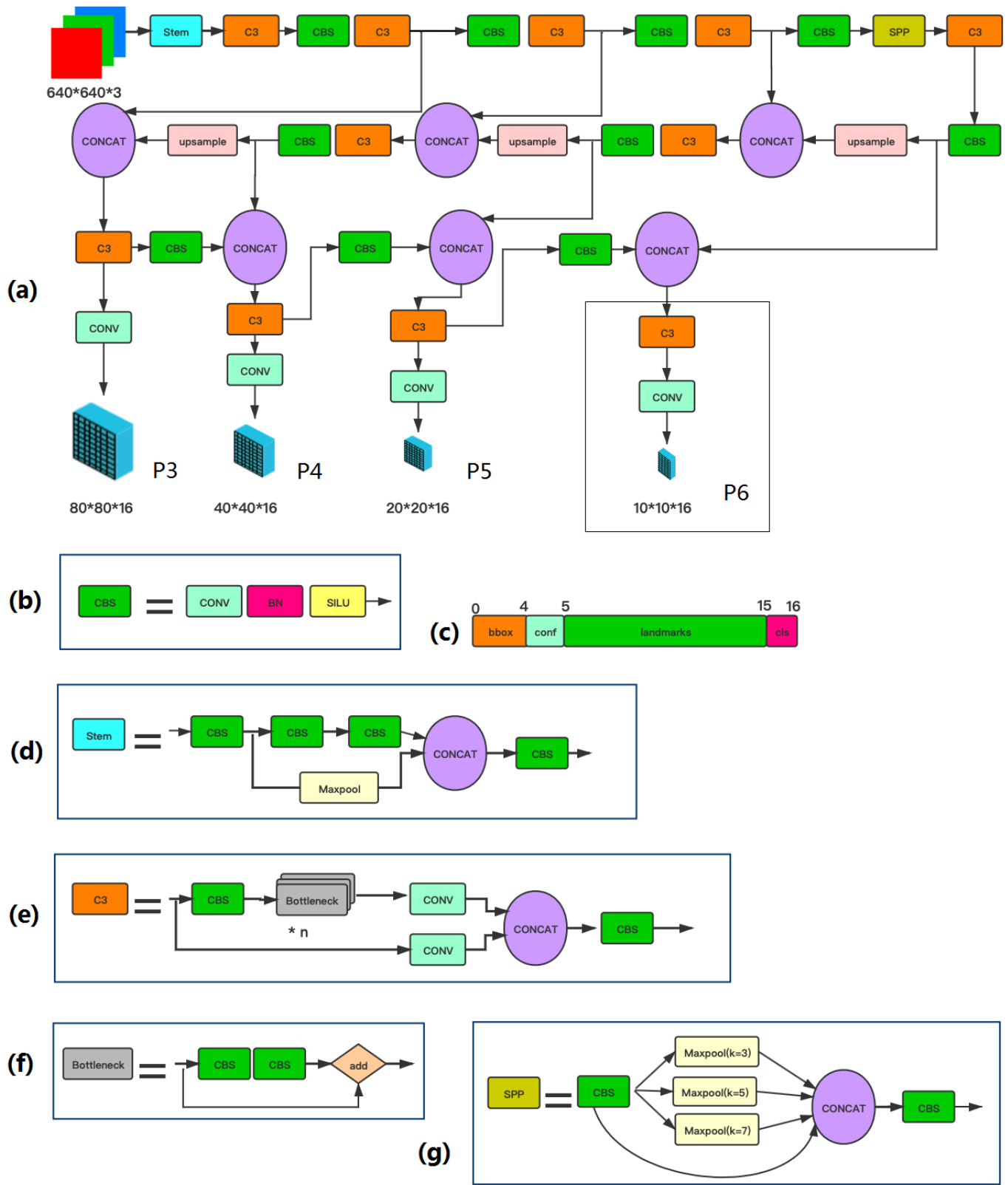


Fig. 1. The proposed YOLO5Face network architecture.

Modification	Method	Easy	Medium	Hard	Params(M)	Flops(G)
Stem block	Focus+Conv	93.56	92.54	82.56	7.091	6.174
	Stem Block	94.13	92.87	82.79	7.075	5.751
SPP Kernel	(13,9,5)	93.43	91.12	82.64	-	-
	(7,5,3)	94.33	92.61	84.15	-	-
P6 block	No	94.31	92.52	83.15	7.075	5.751
	Yes	95.29	93.61	83.13	12.386	6.28
Data augmentation	Baseline (with Mosaic)	91.34	90.21	83.54	-	-
	- up-down flipping	91.87	90.56	83.58	-	-
	+ Ignore small faces	94.12	92.21	82.21	-	-
	+ Random crop	94.34	92.58	83.17	-	-

TABLE II
ABLATION STUDY RESULTS ON THE WIDERFACE VALIDATION DATASET.

Detector	Backbone	Easy	Medium	Hard	Params(M)	Flops(G)
DSFD [28]	ResNet152 [41]	94.29	91.47	71.39	120.06	259.55
RetinaFace [29]	ResNet50 [41]	94.92	91.90	64.17	29.50	37.59
HAMBox [42]	ResNet50 [41]	95.27	93.76	76.75	30.24	43.28
TinaFace [4]	ResNet50 [41]	95.61	94.25	81.43	37.98	172.95
SCRFD-34GF [34]	Bottleneck ResNet	96.06	94.92	85.29	9.80	34.13
SCRFD-10GF [34]	Basic ResNet [41]	95.16	93.87	83.05	3.86	9.98
Our YOLOv5s	YOLOv5-CSPNet [5]	94.33	92.61	83.15	7.075	5.751
Our YOLOv5s6	YOLOv5-CSPNet	95.48	93.66	82.8	12.386	6.280
Our YOLOv5m	YOLOv5-CSPNet	95.30	93.76	85.28	21.063	18.146
Our YOLOv5m6	YOLOv5-CSPNet	95.66	94.1	85.2	35.485	19.773
Our YOLOv5l	YOLOv5-CSPNet	95.9	94.4	84.5	46.627	41.607
Our YOLOv5l6	YOLOv5-CSPNet	96.38	94.90	85.88	76.674	45.279
Our YOLOv5x6	YOLOv5-CSPNet	96.67	95.08	86.55	141.158	88.665
SCRFD-2.5GF [34]	Basic Resnet	93.78	92.16	77.87	0.67	2.53
SCRFD-0.5GF [34]	Depth-wise Conv	90.57	88.12	68.51	0.57	0.508
RetinaFace [29]	MobileNet0.25 [43]	87.78	81.16	47.32	0.44	0.802
FaceBoxes [26]	-	76.17	57.17	24.18	1.01	0.275
Our YOLOv5n	ShuffleNetv2 [6]	93.61	91.54	80.53	1.726	2.111
Our YOLOv5n0.5	ShuffleNetv2-0.5 [6]	90.76	88.12	73.82	0.447	0.571

TABLE III
COMPARISON OF OUR YOLO5FACE AND EXISTING FACE DETECTORS ON THE WIDERFACE VALIDATION DATASET [1].

to aim at generating a fixed-size output irrespective of the input size. In addition, SPP also helps to extract important features by pooling multi-scale versions of itself.

In YOLO5, three kernel sizes 13x13, 9x9, 5x5 are used [5]. We revise them to use smaller size kernels 7x7, 5x5 and 3x3. These smaller kernels help to detect small faces more easily, and increase the overall face detection performance.

F. P6 Output Block

The backbone of YOLO object detector has many layers. As the feature becomes more and more abstract as the layers go deeper, the spatial resolution of feature maps decreases due to downsampling, which leads to a loss of spatial information as well as fine-grained features. In order to preserve these fine-grained features, the FPN [19] is introduced to YOLOv3 [18].

In FPN [19], the fine-grained features take a long path traveling from low-level to high-level layers. To overcome this problem, the PAN is proposed to add a bottom-up augmentation path along the top-down path used in FPN. In addition, in the connection of the feature maps to the lateral architecture, the element-wise addition operation is replaced with concatenation. In FPN, object predictions are done independently on different scale levels, which do not utilize information from other feature maps, and may produce duplicated predictions. In PAN [36], the output feature maps of

bottom-up augmentation pyramid are fused by using (Region of Interest) ROI align and fully connected layers with element-wise max operation.

In YOLOv5, there are three output blocks in the PAN output feature maps, called P3, P4, P5 corresponding to 80x80x16, 40x40x16, 20x20x16, with strides 8, 16, 32, respectively. In our YOLO5Face, we add an extra P6 output block, whose feature map is 10x10x16 with stride 64. This modification particularly helps the detection of large faces. While almost all face detectors focus on improving detection of small faces, detection of large faces can be easily overlooked. We fill this hole by adding the P6 output block.

G. ShuffleNetV2 as Backbone

The ShuffleNet [44] is an extremely efficient CNN for mobile device. The key block is called the ShuffleNet block. It utilizes two new operations, pointwise group convolution and channel shuffle, to greatly reduce computation cost while maintaining accuracy.

The ShuffleNetv2 [44] is an improved version of ShuffleNet. It borrows the shortcut network architecture similar to the DenseNet [39], and the element wise addition is changed to concatenation, similar to the change in PAN [36] in YOLOv5 [5]. But different from DenseNet, ShuffleNetV2 does not densely concatenate, and after the concatenation, the

channel shuffling is used to mix the features. This makes the ShuffleNetV2 a super fast network.

We use the ShuffleNetV2 as the backbone in YOLOv5 and implement super small face detectors YOLOv5n-Face, and YOLOv5n0.5-Face.

IV. EXPERIMENTS

A. Dataset

The WiderFace dataset [1] is the largest face detection dataset, which contains 32,203 images and 393,703 faces. For its large variety of scale, pose, occlusion, expression, illumination and event, it is close to reality and is very challenging.

The whole dataset is divided into train/validation/test sets by ratio 50%/10%/40% within each event class. Furthermore, each subset is defined into three levels of difficulty: Easy, Medium, and Hard. As its names indicate, the Hard subset is most challenging. So the performance on the Hard subset reflects best the effectiveness of a face detector.

Unless specified otherwise, the WiderFace dataset [1] is used in this work. In the face recognition with YOLO5Face landmark and alignment, the Webface dataset [45] is used. The FDDB dataset [46] is used in testing to demonstrate our model's performance on cross-domain datasets.

B. Implementation Details

We use the YOLOv5-4.0 codebase [5] as our starting point and implement all the modifications we describe earlier in PyTorch.

The SGD optimizer is used. The initial learning rate is $1E-2$, the final learning rate is $1E-5$, and the weight decay is $5E-3$. A momentum of 0.8 is used in the first three warming-up epochs. After that, the momentum is changed to 0.937. The training runs 250 epochs with a batch size of 64. The $\lambda_L = 0.5$ is optimized by exhaust search.

Implemented Models. We implement a series of face detector models, as listed in Table I. We implement eight relatively large models, including extra large-size models (YOLOv5x, YOLOv5x6), large-size models (YOLOv5l, YOLOv5l6) medium-size models (YOLOv5m, YOLOv5m6), and small-size models (YOLOv5s, YOLOv5s6). In the name of the model, the last postfix 6 means it has the P6 output block in the SPP. These models all use the YOLOv4 CSPNet as the backbone with different depth and width multiples, denoted as D and W in Table I.

Furthermore, we implement two super small-size models, YOLOv5n and YOLOv5n0.5, which use the ShuffleNetv2 and ShuffleNetv2-0.5 [6] as the backbone. Except for the backbone, all other main blocks, including the stem block, SPP, PAN, are the same as in the larger models.

The number of parameters and number of flops of all these models is listed in Table III for comparison with existing methods.

FaceDetect	training dataset	FNMR
RetinaFace [29]	WiderFace [1]	0.1065
YOLOv5s	WiderFace	0.1060
YOLOv5s	+Multi-task facial [47]	0.1058
YOLOv5m	WiderFace	0.1056
YOLOv5m	+Multi-task facial	0.1051

TABLE IV

EVALUATION OF YOLO5FACE LANDMARK ON FACE RECOGNITION ON THE WEBFACE TEST DATASET [45].

C. Ablation Study

In this subsection we present the effects of the modifications we have in our YOLO5Face. In this study we use the YOLO5s model. We use the WiderFace [1] validation dataset and use the mAP as the performance metric.

Stem Block vs. Focus Layer. The mAP performances of the stem block [38] and the focus layer are listed in first panel of Table II. Also listed are the number of parameters and number of flops. From the results we see that the stem block improves the mAP by 0.57%, 0.33%, and 0.23% on the easy, medium, and hard subset, respectively.

SPP with Smaller Size Kernels. The mAP performances of the SPP [35] kernel sizes (7x7, 5x5, 3x3) and original kernel sizes (13x13, 9x9, 5x5) are listed in the second panel of Table II. From the results we see that the smaller kernel sizes improve the mAP by 0.9%, 1.49%, and 1.41% on the easy, medium, and hard subset, respectively. The improvements are larger than that from the Stem block [38].

P6 Output Block. The mAP performances of the addition of the P6 output block are listed in the third panel of Table II. From the results we see that the P6 block improves the mAP by 0.98%, 1.09%, and -0.02% on the easy, medium, and hard subset, respectively.

Data Augmentation Performance results of a few data augmentation methods are listed in the fourth panel of Table II. From the results we see that ignoring small faces, random crop help the mAP in the Easy and Medium dataset, while the Mosaic [3] helps the mAP in the Hard dataset. As we explain before, the Mosaic has to work with the ignoring small faces, otherwise the performance degrades dramatically.

Please note that in these experiments the network configurations are not incremental. However in each of set of experiment, the baselines for the two networks are the same to make the comparison fair. For example in the SPP experiments, except for the kernel sizes are different, all other settings are identical.

D. YOLO5Face for Face Recognition

Landmark is critical for face recognition accuracy. In RetinaFace [29], the accuracy of the landmark is evaluated with the MSE between estimated landmark coordinates and their ground truth and with the face recognition accuracy. The results show that the RetinaFace has better landmarks than the older MTCNN [25].

In this work, we also use face recognition to evaluate the accuracy of landmarks of the YOLO5Face. We use the Webface test dataset, which is the largest face dataset with noisy 4M

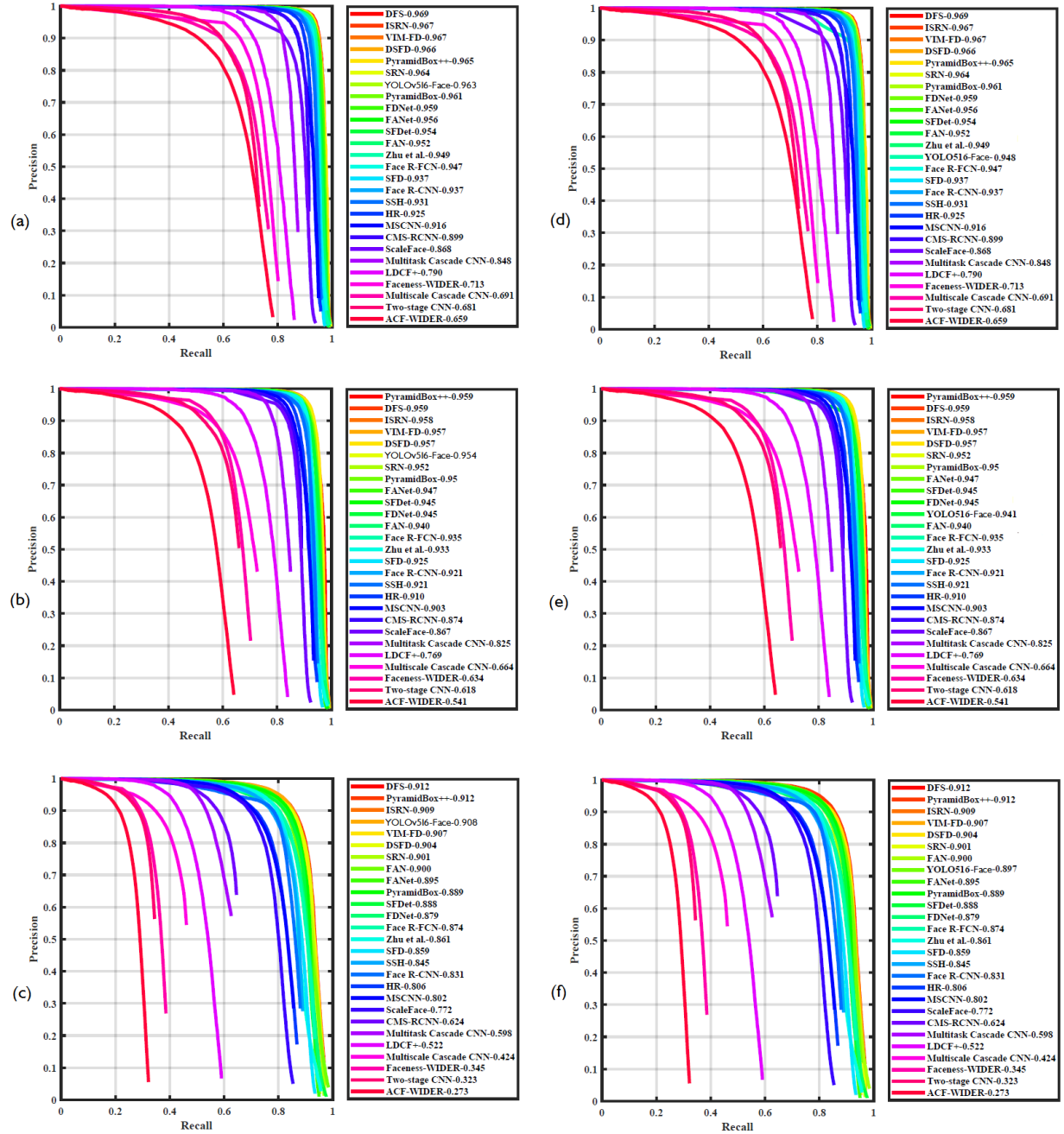


Fig. 2. The precision-recall (PR) curves of face detectors, (a) validation-Easy, (b) validation-Medium, (c) validation-Hard, (d) test-Easy, (e) test-Medium, (f) test-Hard.

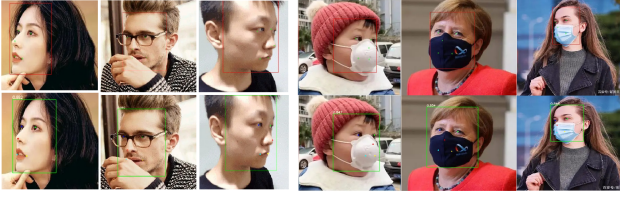


Fig. 3. Some examples of detected face and landmarks, where the first row is from RetinaFace [29], and second row is from our YOLOv5m.

identities/260M faces, and cleaned 2M identities/42M faces [45]. This dataset is used in the ICCV2021 Masked Face Recognition (MFR) challenge [48]. In this challenge, both masked face images and standard face images are included, and a metric False Non-Match Rate (FNMR) at False Match Rate (FMR) = $1E-5$ is used. The $FNMR * 0.25$ for MFR plus $FNMR * 0.75$ for standard face recognition is combined as the final metric.

By default, the RetinaFace [49] is used as the face detector on the dataset. We compare our YOLO5Face with the RetinaFace on this dataset. We use ArcFace [50] framework with Resnet124 [41] as backbone. Extracted features of two models trained on the Glint360k dataset [51] are concatenated as the baseline model. We replace the RetinaFace with our YOLO5Face. We test two models, a small model YOLOv5s, and a medium model YOLOv5m. More details can be found in [52].

The results are listed in Table IV. From the results, we see that both our small and medium models outperform the RetinaFace [29]. In addition, we notice that there are very few large face images in the WiderFace dataset, so we add some large face images from the Multi-task-facial dataset [47] into the YOLO5Face training dataset. We find that this technique improves face recognition performance.

shown in Figure 3 are some detected Webface [45] faces and landmarks using the RetinaFace [29] and our YOLOv5m. On the faces of a large pose, we can visually observe that our landmarks are more accurate, which has been proved in our face recognition results shown in Table IV.

E. YOLO5Face on WiderFace Dataset

We compare our YOLO5Face with many existing face detectors on the WiderFace dataset. The results are listed in Table III, where the previous SOTA results and our best results are both highlighted.

We first look at the performance of relatively large models whose number of parameters is larger than 3M and the number of flops is larger than 5G. All existing methods achieve mAP in 94.27-96.06% on the Easy subset, 91.9-94.92% on the Medium subset, and 71.39-85.29% on the Hard subset. The most recently released SCRFD [34] achieves the best performance in all subsets. Our YOLO5Face (YOLOv5x6) achieves 96.67%, 95.08%, 86.55% on the three subsets, respectively. We achieve the SOTA performance on all the Easy, Medium, and Hard subsets.

Method	MAP
ASFD [31]	0.9911
RefineFace [30]	0.9911
PyramidBox [58]	0.9869
FaceBoxes [26]	0.9598
Our YOLOv5s	0.9843
Our YOLOv5m	0.9849
Our YOLOv5l	0.9867
Our YOLOv5l6	0.9880

TABLE V

EVALUATION OF YOLO5FACE ON THE FDDB DATASET [46].

Next, we look at the performance of super small models whose number of parameters is less than 2M and the number of flops is less than 3G. All existing methods achieve mAP in 76.17-93.78% on the Easy subset, 57.17-92.16% on the Medium subset, and 24.18-77.87% on the Hard subset. Again, the SCRFD [34] achieves the best performance in all subsets. Our YOLO5Face (YOLOv5n) achieves 93.61%, 91.54%, 80.53% on the three subsets, respectively. Our face detector has a little bit worse performance than the SCRFD [34] on the Easy and Medium subsets. However, on the Hard subset, our face detector is leading by 2.66%. Furthermore, our smallest model, YOLOv5n0.5, has good performance, even its model size is much smaller.

The precision-recall (PR) curves of our YOLO5Face face detector, along with the competitors, are shown in Figure 2. The leading competitors include DFS [53], ISRN [54], VIM-FD [55], DSFD [28], PyramidBox++ [56], SRN [57], PyramidBox [58] and more. For a full list of the competitors and their results on the WiderFace [1] validation and test datasets, please refer to [2]. In the results on the validation dataset, our YOLOv5x6-Face detector achieves 96.9%, 96.0%, 91.6% mAP on the Easy, Medium, and Hard subset, respectively, exceeding the previous SOTA by 0.0%, 0.1%, 0.4%. In the results on the test dataset, our YOLOv5x6-Face detector achieves 95.8%, 94.9%, 90.5% mAP on the Easy, Medium, and Hard subset, respectively with 1.1%, 1.0%, 0.7% gap to the previous SOTA. Please note that, in these evaluations, we only use multiple scales and left-right flipping without using other test-time augmentation (TTA) methods. Our focus is more on the VGA input images, where we achieve the SOTA in almost all conditions.

F. YOLO5Face on FDDB Dataset

FDDB dataset [46] is a small dataset with 5171 faces annotated in 2845 images. To demonstrate our YOLO5Face's performance on the cross-domain dataset, we test it on the FDDB dataset without retraining on it. The performances of true positive rate (TPR) when the number of false-positive is 1000 are listed in Table 4. Please note that it is pointed out in RefineFace [30] that the annotation of FDDB misses many faces. In order to achieve their performance of 0.9911, the RefineFace modifies the FDDB annotation. In our evaluation, we use the original FDDB annotation without modifications. RetinaFace [29] is not evaluated on the FDDB dataset.

V. CONCLUSION

In this paper we present our YOLO5Face based on YOLOv5 object detector [5]. We implement eight models. Both the largest model YOLOv5l6 and the super small model YOLOv5n achieve close to or exceeding SOTA performance on the WiderFace [1] validation Easy, Medium and Hard subsets. This proves the effectiveness of our YOLO5Face in not only achieving the best performance, but also running fast. Since we open-source the code, a lot of applications and mobile apps have been developed based on our design, and achieve impressive performance.

REFERENCES

- [1] S. Yang, P. Luo, C. C. Loy, and X. Tang, "Wider face: A face detection benchmark," *CVPR*, 2016. 1, 2, 5, 6, 8, 9
- [2] S. Yang, P. Luo, C. C. Loy, and X. Tang, "Wider face: A face detection benchmark," <http://shuoyang1213.me/WIDERFACE/index.html>. 1, 2, 8
- [3] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "Yolov4: Optimal speed and accuracy of object detection," *ArXiv preprint 2004.10934*, 2020. 1, 2, 3, 6
- [4] Y. Zhu, H. Cai, S. Zhang, C. Wang, and W. Xiong, "Tinaface: Strong but simple baseline for face detection," *ArXiv preprint 2011.13183*, 2020. 1, 2, 5
- [5] YOLOv5, "Yolov5," <https://github.com/ultralytics/yolov5>. 1, 2, 3, 5, 6, 9
- [6] M. Ma, X. Zhang, H. Zheng, and J. Sun, "Shufflenet v2: Practical guidelines for efficient cnn architecture design," *ArXiv preprint 1807.11164*, 2018. 1, 3, 5, 6
- [7] Z. Zhang, C. and Zhang, "Robust real-time face detection," *IJCV*, 2004. 1
- [8] M. Yang, D. Kriegman, and N. Ahuja, "Detecting faces in images: a survey," *TPAMI*, 2002. 1
- [9] Z. Zhang, C. and Zhang, "A survey of recent advances in face detection," *Microsoft Research Technical report*, 2010. 1
- [10] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014. 2
- [11] Ross Girshick, "Fast R-CNN," in *Proceedings of the International Conference on Computer Vision (ICCV)*, 2015. 2
- [12] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2016. 2
- [13] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick, "Mask R-CNN," in *Proceedings of the International Conference on Computer Vision (ICCV)*, 2017. 2
- [14] Z. Cai and N. Vasconcelos, "Cascade r-cnn: Delving into high quality object detection," *CVPR*, 2018. 2
- [15] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C. Fu, and A. Berg, "Yolov3: An incremental improvement," *ECCV*, 2016. 2
- [16] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi, "You only look once: Unified, real-time object detection," *CVPR*, 2016. 2
- [17] Joseph Redmon and Ali Farhadi, "Yolo9000: better, faster, stronger," *CVPR*, 2017. 2
- [18] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2015. 2, 5
- [19] T. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," *CVPR*, 2017. 2, 5
- [20] K. et al. Chen, "Mmdetection: Open mmlab detection toolbox and benchmark," *ECCV*, 2020. 2
- [21] M. Tan, R. Pang, and Q. Le, "Efficientdet: Scalable and efficient object detection," *CVPR*, 2020. 2
- [22] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and Z. Zagoruyko, "End-to-end object detection with transformers," *ECCV*, 2020. 2
- [23] K. Duan, S. Bai, L. Xie, H. Qi, Q. Huang, and Q. Tian, "Centernet: Keypoint triplets for object detection," *ICCV*, 2019. 2
- [24] X. Zhou, D. Wang, and Krähnenbühl P., "Objects as points," in *arXiv preprint arXiv:1904.07850*, 2019. 2
- [25] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016. 2, 3, 6
- [26] Shifeng Zhang, Xiangyu Zhu, Zhen Lei, Hailin Shi, Xiaobo Wang, and Stan Z. Li, "Faceboxes: A cpu real-time face detector with high accuracy," *IJCB*, 2017. 2, 5, 8
- [27] S. Zhang, X. Zhu, Z. Lei, H. Shi, X. Wang, and S. Z. Li, "S³fd: Single shot scale-invariant face detector," *ICCV*, 2017. 2
- [28] J. Li, Y. Wang, C. Wang, Y. Tai, J. Qian, J. Yang, C. Wang, J. Li, and F. Huang, "Dsf: Dual shot face detector," *ArXiv preprint 1810.102207*, 2018. 2, 5, 8
- [29] J. Deng, J. Guo, Y. Zhou, J. Yu, I. Kotsia, and S. Zafeiriou, "Retinaface: Single-stage dense face localisation in the wild," *CVPR*, 2020. 2, 5, 6, 8
- [30] S. Zhang, C. Chi, Z. Lei, and S.Z. Li, "Refineface: Refinement neural network for high performance face detection," *ArXiv preprint 1909.04376*, 2019. 2, 8
- [31] B. Zhang, J. Li and Y. Wang, Y. Tai, C. Wang, J. Li, F. Huang, Y. Xia, W. Pei, and R. Ji, "Automatic and scalable face detector," *ArXiv preprint 2003.11228*, 2020. 2, 8
- [32] D. Yashunin, T. Baydasov, and R. Vlasov, "Maskface: multi-task face and landmark detector," *ArXiv preprint 2005.09412*, 2020. 2
- [33] Y. Liu, F. Wang, B. Sun, and H. Li, "Mogface: Rethinking scale augmentation on the face detector," *ArXiv preprint 2103.11139*, 2021. 2
- [34] J. Guo, A. Deng, J. Lattas, and S. Zafeiriou, "Sample and computation redistribution for efficient face detection," *ArXiv preprint 2105.04714*, 2021. 2, 5, 8
- [35] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *TPAMI*, 2015. 2, 3, 6
- [36] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," *ArXiv preprint 1803.01534*, 2018. 2, 5
- [37] Stefan Elfving, Eiji Uchibe, and Kenji Doyab, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *ArXiv preprint 1702.03118*, 2017. 2
- [38] Robert J. Wang, Xiang Li, and Charles X. Ling, "Pelee: A real-time object detection system on mobile devices," *NeurIPS*, 2018. 2, 3, 6
- [39] G. Huang, Z. Liu, L. Maaten, and K.Q. Weinberger, "Densely connected convolutional networks," *CVPR*, 2017. 2, 5
- [40] Z. Feng, J. Kittler, M. Awais, P. Huber, and X. Wu, "Wing loss for robust facial landmark localisation with convolutional neural networks," *CVPR*, 2018. 3
- [41] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CVPR*, 2016. 5, 8
- [42] Yang Liu, Xu Tang, Xiang Wu, Junyu Han, Jingtuo Liu, and Errui Ding, "Hambox: Delving into online high-quality anchors mining for detecting outer faces," *CVPR*, 2020. 5
- [43] M. Sandler, A. Howard, w. Zhu, A. Zhmoginov, and L. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," *CVPR*, 2018. 5
- [44] X. Zhang, X. Zhou, M. Lin, and J. Sun, "Shufflenet: An extremely efficient convolutional neural network for mobile devices," *ArXiv preprint 1707.01083*, 2017. 5
- [45] Z. Zhu, G. Huang, J. Deng, Y. Ye, J. Huang, X. Chen, J. Zhu, T. Yang, J. Lu, D. Du, and J. Zhou, "Webface260m: A benchmark unveiling the power of million-scale deep face recognition," *CVPR*, 2021. 6, 8
- [46] V. Jain and E. Learned-Miller, "Fddb: A benchmark for face detection in unconstrained settings," *University of Massachusetts Report*, no. UM-CS-2010-009, 2010. 6, 8
- [47] Rui Zhao, Tianshan Liu, Jun Xiao, Daniel P. K. Lun, and Kin-Man Lam, "Deep multi-task learning for facial expression recognition and synthesis based on selective feature sharing," *ICPR*, 2020. 6, 8
- [48] Zheng Zhu, Guan Huang, Jiankang Deng, Yun Ye, Junjie Huang, Xinze Chen, Jiagang Zhu, Tian Yang, Jia Guo, Jiwen Lu, Dalong Du, and Jie Zhou, "Masked face recognition challenge: The webface260m track report," *ICCV Workshops*, 2021. 8
- [49] T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li, "Bag of tricks for image classification with convolutional neural networks," *CVPR*, 2019. 8
- [50] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "Arcface: Additive angular margin loss for deep face recognition," *CVPR*, June 2019. 8

- [51] X. An, X. Zhu, Y. Xiao, L. Wu, M. Zhang, Y. Gao, B. Qin, D. Zhang, and Y. Fu, "Partial fc: Training 10 million identities on a single machine," *arXiv preprint 2010.05222*, 2021. 8
- [52] Delong Qi, Kangli Hu, Weijun Tan, Qi Yao, and Jingfeng Liu, "Balanced masked and standard face recognition," *ICCV Workshops*, 2021. 8
- [53] W. Tian, Z. Wang, H. Shen, W. Deng, B. Chen, and X. Zhang, "Learning better features for face detection with feature fusion and segmentation supervision," *ArXiv preprint 1811.08557*, 2018. 8
- [54] S. Zhang, R. Zhu, X. Wang, H. Shi, T. Fu, S. Wang, T. Mei, and Stan Z. Li, "Isrn - improved selective refinement network for face detection," *ArXiv preprint 1901.06651*, 2019. 8
- [55] Y. Zhang, X. Xu, and X. Liu, "Robust and high performance face detector," *ArXiv preprint 1901.02350*, 2019. 8
- [56] Z. Li, X. Tang, J. Han, J. Liu, and Z. He, "Pyramidbox++: High performance detector for finding tiny face," *ArXiv preprint 1904.00386*, 2019. 8
- [57] C. Chi, S. Zhang, J. Xing, Z. Lei, and S. Z. Li, "Srn - selective refinement network for high performance face detection," *ArXiv preprint 1809.02693*, 2018. 8
- [58] X. Tang, Daniel K. Du, Z. He, and J. Liu, "Pyramidbox: A context-assisted single shot face detector," *ArXiv preprint 1803.07737*, 2018. 8